

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\solvers\LaguerreSolver.java

- Specification

module LaguerreSolver

type

min, max, start: real // interval bounds and (optional) start value
 maxEval: integer // maximum function evaluations
 relAcc, absAcc, fAcc: real // solver tolerances: relative, absolute, function-value
 r: real

process solvePF(c[], min, max, start, maxEval, relAcc, absAcc, fAcc: real) r: real

functional scenario form:

```

c = 0 && ex = NullArgument
||
min > max && ex = BadSequence
||
bracketed = 0 && ex = NoBracketing
||
yMin = 0 && ex = None and r = min
||
yMax = 0 && ex = None and r = max
||
yStart = 0 && ex = None and r = start
||
yStart * yMin < 0 && ex = None
    and r ∈ (min, start] and |f(r)| <= fAcc
||
yStart * yMax < 0 && ex = None
    and r ∈ [start, max) and |f(r)| <= fAcc
||
    Im <= tol
    && ex = None
    and r = Re(z) and |f(r)| <= fAcc
||
re <= fAcc
    && ex = None
    and r = Re(root[i]) and |f(r)| <= fAcc
||
bracketed=1 && ex = None and r = NaN
||

```

evaluations > maxEval && ex = TooManyEvaluations

end process

end module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.solvers;
import org.apache.commons.numbers.complex.Complex;
import org.apache.commons.math4.legacy.analysis.polynomials.PolynomialF
unction;
import org.apache.commons.math4.legacy.exception.NoBracketingException;
import org.apache.commons.math4.legacy.exception.NoDataException;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsExce
ption;
import org.apache.commons.math4.legacy.exception.util.LocalizedFormats;
//import org.apache.commons.math4.legacy.optim.Linear.VT;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
/**
 * Implements the <a href="http://mathworld.wolfram.com/LaguerresMethod
.html">
```

```

* Laguerre's Method for root finding of real coefficient polynomials. For
* reference, see <blockquote> <b>A First Course in Numerical Analysis</b>
/b>, ISBN
* 048641454X, chapter 8. </blockquote> Laguerre's method is global in
the sense
* that it can start with any initial approximation and be able to solve all
* roots from that point. The algorithm requires a bracketing condition
.
*
* @since 1.2
*/
public class LaguerreSolver extends AbstractPolynomialSolver {
    /** Default absolute accuracy. */
    private static final double DEFAULT_ABSOLUTE_ACCURACY = 1e-6;
    /** Complex solver. */
    private final ComplexSolver complexSolver = new ComplexSolver();
    /**
     * Construct a solver with default accuracy (1e-6).
     */
    public LaguerreSolver() {
        this(DEFAULT_ABSOLUTE_ACCURACY);
    }
    /**
     * Construct a solver.
     *
     * @param absoluteAccuracy Absolute accuracy.
     */
    public LaguerreSolver(double absoluteAccuracy) {
        super(absoluteAccuracy);
    }
    /**
     * Construct a solver.
     *
     * @param relativeAccuracy Relative accuracy.
     * @param absoluteAccuracy Absolute accuracy.
     */
    public LaguerreSolver(double relativeAccuracy, double absoluteAccuracy) {
        super(relativeAccuracy, absoluteAccuracy);
    }
    /**
     * Construct a solver.

```

```

*
* @param relativeAccuracy      Relative accuracy.
* @param absoluteAccuracy      Absolute accuracy.
* @param functionValueAccuracy Function value accuracy.
*/
public LaguerreSolver(double relativeAccuracy, double absoluteAccuracy
, double functionValueAccuracy) {
    super(relativeAccuracy, absoluteAccuracy, functionValueAccuracy);
}
/**
 * {@inheritDoc}
 */
@Override
public double doSolve() throws TooManyEvaluationsException, NumberIsToo
oLargeException, NoBracketingException {
    final double min = getMin();
    final double max = getMax();
    final double initial = getStartValue();
    final double functionValueAccuracy = getFunctionValueAccuracy();
    verifySequence(min, initial, max);
    // Return the initial guess if it is good enough.
    final double yInitial = computeObjectiveValue(initial);
    if (JdkMath.abs(yInitial) <= functionValueAccuracy) {
        return initial;
    }
    // Return the first endpoint if it is good enough.
    final double yMin = computeObjectiveValue(min);
    if (JdkMath.abs(yMin) <= functionValueAccuracy) {
        return min;
    }
    // Reduce interval if min and initial bracket the root.
    if (yInitial * yMin < 0) {
        return laguerre(min, initial);
    }
    // Return the second endpoint if it is good enough.
    final double yMax = computeObjectiveValue(max);
    if (JdkMath.abs(yMax) <= functionValueAccuracy) {
        return max;
    }
    // Reduce interval if initial and max bracket the root.
    if (yInitial * yMax < 0) {
        return laguerre(initial, max);
    }
    throw new NoBracketingException(min, max, yMin, yMax);
}

```

```

}
/**
 * Find a real root in the given interval.
 *
 * Despite the bracketing condition, the root returned by
 * {@link LaguerreSolver.ComplexSolver#solve(Complex[],Complex)} may not be a
 * real zero inside {@code [min, max]}. For example,
 * p(x) = x3 + 1, with min = -2,
 * max = 2, initial = 0. When it occurs, this code calls
 * {@link LaguerreSolver.ComplexSolver#solveAll(Complex[],Complex)} in
 * order to
 * obtain all roots and picks up one real root.
 *
 * @param lo Lower bound of the search interval.
 * @param hi Higher bound of the search interval.
 * @return the point at which the function value is zero.
 */
private double laguerre(double lo, double hi) {
    final Complex[] c = real2Complex(getCoefficients());
    final Complex initial = Complex.ofCartesian(0.5 * (lo + hi), 0);
    final Complex z = complexSolver.solve(c, initial);
    if (complexSolver.isRoot(lo, hi, z)) {
        return z.getReal();
    } else {
        double r = Double.NaN;
        // Solve all roots and select the one we are seeking.
        Complex[] root = complexSolver.solveAll(c, initial);
        for (int i = 0; i < root.length; i++) {
            if (complexSolver.isRoot(lo, hi, root[i])) {
                r = root[i].getReal();
                break;
            }
        }
        return r;
    }
}
/**
 * Find all complex roots for the polynomial with the given coefficients,
 * starting from the given initial value.
 * <p>

```

```

* Note: This method is not part of the API of {@link BaseUnivariateSo
lver}.
* </p>
*
* @param coefficients Polynomial coefficients.
* @param initial      Start value.
* @return the point at which the function value is zero.
* @throws org.apache.commons.math4.legacy.exception.TooManyEvaluation
sException if
*
*     the
*
*     maximum
*
*     number
*
*     of
*
*     evaluations
*
*     is
*
*     exceeded.
* @throws NullPointerException
*     if
*
*     the
*
*     {@code coefficients}
*
*     is
*
*     {@code null}.
* @throws NoDataException
*     if
*
*     the
*
*     {@code coefficients}
*
*     array
*
*     is

```

```

*
    empty.
* @since 3.1
*/
public Complex[] solveAllComplex(double[] coefficients, double initial
)
    throws NullPointerException, NoDataException, TooManyEvaluationsExc
eption {
    setup(Integer.MAX_VALUE, new PolynomialFunction(coefficients), Double
.NEGATIVE_INFINITY,
        Double.POSITIVE_INFINITY, initial);
    return complexSolver.solveAll(real2Complex(coefficients), Complex.ofC
artesian(initial, 0d));
}
/**
 * Find a complex root for the polynomial with the given coefficients,
 * starting
 * from the given initial value.
 * <p>
 * Note: This method is not part of the API of {@link BaseUnivariateSo
lver}.
 * </p>
 *
 * @param coefficients Polynomial coefficients.
 * @param initial      Start value.
 * @return the point at which the function value is zero.
 * @throws org.apache.commons.math4.legacy.exception.TooManyEvaluation
sException if
 *
 * the
 *
 * maximum
 *
 * number
 *
 * of
 *
 * evaluations
 *
 * is
 *
 * exceeded.
 * @throws NullPointerException
 * if

```

```

*
    the
*
    {@code coefficients}
*
    is
*
    {@code null}.
* @throws NoDataException
    if
*
    the
*
    {@code coefficients}
*
    array
*
    is
*
    empty.
* @since 3.1
*/
public Complex solveComplex(double[] coefficients, double initial)
    throws NullPointerException, NoDataException, TooManyEvaluationsExc
    eption {
    setup(Integer.MAX_VALUE, new PolynomialFunction(coefficients), Double
        .NEGATIVE_INFINITY,
        Double.POSITIVE_INFINITY, initial);
    return complexSolver.solve(real2Complex(coefficients), Complex.ofCart
        esian(initial, 0d));
}
/**
 * Class for searching all (complex) roots.
 */
private class ComplexSolver {
    /**
     * Check whether the given complex root is actually a real zero in th
     e given
     * interval, within the solver tolerance level.
     *
     * @param min Lower bound for the interval.
     * @param max Upper bound for the interval.
     * @param z Complex root.
     * @return {@code true} if z is a real zero.

```

```

    */
    public boolean isRoot(double min, double max, Complex z) {
        if (isSequence(min, z.getReal(), max)) {
            double tolerance = JdkMath.max(getRelativeAccuracy() * z.abs(), get
AbsoluteAccuracy());
            return JdkMath.abs(z.getImaginary()) <= tolerance || z.abs() <= get
FunctionValueAccuracy();
        }
        return false;
    }
}
/**
 * Find all complex roots for the polynomial with the given coefficie
nts,
 * starting from the given initial value.
 *
 * @param coefficients Polynomial coefficients.
 * @param initial      Start value.
 * @return the point at which the function value is zero.
 * @throws org.apache.commons.math4.legacy.exception.TooManyEvaluatio
nsException if
 *
 * the
 *
 * maximum
 *
 * number
 *
 * of
 *
 * evaluations
 *
 * is
 *
 * exceeded.
 * @throws NullPointerException
 * if
 *
 * the
 *
 * {@code coefficients}
 *
 * is
 *
 * {@code null}.

```

```

* @throws NoDataException
    if
*
    the
*
    {@code coefficients}
*
    array
*
    is
*
    empty.
*/
public Complex[] solveAll(Complex[] coefficients, Complex initial)
    throws NullArgumentException, NoDataException, TooManyEvaluationsEx
ception {
    if (coefficients == null) {
        throw new NullArgumentException();
    }
    final int n = coefficients.length - 1;
    if (n == 0) {
        throw new NoDataException(LocalizedFormats.POLYNOMIAL);
    }
    // Coefficients for deflated polynomial.
    final Complex[] c = coefficients.clone();
    // Solve individual roots successively.
    final Complex[] root = new Complex[n];
    for (int i = 0; i < n; i++) {
        final Complex[] subarray = new Complex[n - i + 1];
        System.arraycopy(c, 0, subarray, 0, subarray.length);
        root[i] = solve(subarray, initial);
        // Polynomial deflation using synthetic division.
        Complex newc = c[n - i];
        Complex oldc = null;
        for (int j = n - i - 1; j >= 0; j--) {
            oldc = c[j];
            c[j] = newc;
            newc = oldc.add(newc.multiply(root[i]));
        }
    }
    return root;
}
/**

```

```

    * Find a complex root for the polynomial with the given coefficients
, starting
    * from the given initial value.
    *
    * @param coefficients Polynomial coefficients.
    * @param initial      Start value.
    * @return the point at which the function value is zero.
    * @throws org.apache.commons.math4.legacy.exception.TooManyEvaluatio
nsException if
    *
    * the
    *
    * maximum
    *
    * number
    *
    * of
    *
    * evaluations
    *
    * is
    *
    * exceeded.
    * @throws NullPointerException
    * if
    *
    * the
    *
    * {@code coefficients}
    *
    * is
    *
    * {@code null}.
    * @throws NoDataException
    * if
    *
    * the
    *
    * {@code coefficients}
    *
    * array
    *
    * is

```

```

*
    empty.
*/
public Complex solve(Complex[] coefficients, Complex initial)
    throws NullPointerException, NoDataException, TooManyEvaluationsEx
ception {
    if (coefficients == null) {
        throw new NullPointerException();
    }
    final int n = coefficients.length - 1;
    if (n == 0) {
        throw new NoDataException(LocalizedFormats.POLYNOMIAL);
    }
    final double absoluteAccuracy = getAbsoluteAccuracy();
    final double relativeAccuracy = getRelativeAccuracy();
    final double functionValueAccuracy = getFunctionValueAccuracy();
    final Complex nC = Complex.ofCartesian(n, 0);
    final Complex n1C = Complex.ofCartesian(n - 1, 0);
    Complex z = initial;
    Complex oldz = Complex.ofCartesian(Double.POSITIVE_INFINITY, Double.
POSITIVE_INFINITY);
    while (true) {
        // Compute pv (polynomial value), dv (derivative value), and
        // d2v (second derivative value) simultaneously.
        Complex pv = coefficients[n];
        Complex dv = Complex.ZERO;
        Complex d2v = Complex.ZERO;
        for (int j = n - 1; j >= 0; j--) {
            d2v = dv.add(z.multiply(d2v));
            dv = pv.add(z.multiply(dv));
            pv = coefficients[j].add(z.multiply(pv));
        }
        d2v = d2v.multiply(2);
        // Check for convergence.
        final double tolerance = JdkMath.max(relativeAccuracy * z.abs(), ab
soluteAccuracy);
        if ((z.subtract(oldz)).abs() <= tolerance) {
            return z;
        }
        if (pv.abs() <= functionValueAccuracy) {
            return z;
        }
        // Now pv != 0, calculate the new approximation.
        final Complex g = dv.divide(pv);

```

```

        final Complex g2 = g.multiply(g);
        final Complex h = g2.subtract(d2v.divide(pv));
        final Complex delta = n1C.multiply((nC.multiply(h)).subtract(g2));
        // Choose a denominator larger in magnitude.
        final Complex deltaSqrt = delta.sqrt();
        final Complex dplus = g.add(deltaSqrt);
        final Complex dminus = g.subtract(deltaSqrt);
        final Complex denominator = dplus.abs() > dminus.abs() ? dplus : dminus;
        // Perturb z if denominator is zero, for instance,
        //  $p(x) = x^3 + 1$ ,  $z = 0$ .
        // This uses exact equality to zero. A tolerance may be required here.
        if (denominator.equals(Complex.ZERO)) {
            z = z.add(Complex.ofCartesian(absoluteAccuracy, absoluteAccuracy));
        }
        oldz = Complex.ofCartesian(Double.POSITIVE_INFINITY, Double.POSITIVE_INFINITY);
    } else {
        oldz = z;
        z = z.subtract(nC.divide(denominator));
    }
    incrementEvaluationCount();
}
}
}
/**
 * Converts a {@code double[]} array to a {@code Complex[]} array.
 *
 * @param real array of numbers to be converted to their {@code Complex}
 *           equivalent
 * @return {@code Complex} array
 */
private static Complex[] real2Complex(double[] real) {
    int index = 0;
    final Complex[] c = new Complex[real.length];
    for (final double d : real) {
        c[index] = Complex.ofCartesian(d, 0);
        index++;
    }
    return c;
}
}

```

```

//VT
class VT {
    // Build PolynomialFunction from real roots:  $\prod (x - r_i)$ 
    private static org.apache.commons.math4.legacy.analysis.polynomials.Po
    lynomialFunction
    polyFromRoots(double... roots) {
        // coefficients in ascending powers ( $c_0 + c_1 x + c_2 x^2 + \dots$ )
        double[] coeff = new double[]{1.0}; // start with constant 1
        for (double r : roots) {
            double[] next = new double[coeff.length + 1];
            // next[0] = -r * c0
            next[0] = -r * coeff[0];
            // middle terms: next[k] = c_{k-1} - r * c_k for k = 1..n-1
            for (int k = 1; k < next.length - 1; k++) {
                double ck = (k < coeff.length) ? coeff[k] : 0.0;
                double ckm1 = coeff[k - 1];
                next[k] = ckm1 - r * ck;
            }
            // highest term: c_n
            next[next.length - 1] = coeff[coeff.length - 1];
            coeff = next;
        }
        return new org.apache.commons.math4.legacy.analysis.polynomials.Po
    lynomialFunction(coeff);
    }
    private static boolean approx(double a, double b, double tol) {
        if (Double.isNaN(a) || Double.isNaN(b)) return false;
        if (Double.isInfinite(a) || Double.isInfinite(b)) return a == b;
        return Math.abs(a - b) <= tol * Math.max(1.0, Math.max(Math.abs(a)
, Math.abs(b)));
    }
    private static void check(String name, boolean cond) {
        System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
    }
    public void Vtest() {
        System.out.println("=== LaguerreSolver embedded tests (PolynomialF
unction) ===");
        LaguerreSolver solver = new LaguerreSolver(); // default tolerance
        s
        // VT1: cubic (x-1)(x-2)(x-3) in [0.5, 1.5] → root ~ 1
        try {
            org.apache.commons.math4.legacy.analysis.polynomials.PolynomialF
            unction f =
                polyFromRoots(1.0, 2.0, 3.0);

```

```

        double r = solver.solve(1000, f, 0.5, 1.5);
        System.out.printf("VT1 root=%.12f, f(root)=%.3e\n", r, f.value
(r));
        check("VT1 ~ 1.0", approx(r, 1.0, 1e-10));
    } catch (Throwable t) { t.printStackTrace(); check("VT1 unexpected
exception", false); }
    // VT2: same cubic, interval targets the middle root ~ 2
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 2.0, 3.0);
        double r = solver.solve(1000, f, 1.5, 2.5);
        System.out.printf("VT2 root=%.12f, f(root)=%.3e\n", r, f.value
(r));
        check("VT2 ~ 2.0", approx(r, 2.0, 1e-10));
    } catch (Throwable t) { t.printStackTrace(); check("VT2 unexpected
exception", false); }
    // VT3: right endpoint is root: (x-4) on [2,4] → 4
    try {
        // (x-4) => coefficients: [-4, 1]
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
                new double[]{-4.0, 1.0});
        double r = solver.solve(1000, f, 2.0, 4.0);
        System.out.printf("VT3 root=%.12f\n", r);
        check("VT3 == 4.0", approx(r, 4.0, 0));
    } catch (Throwable t) { t.printStackTrace(); check("VT3 unexpected
exception", false); }
    // VT4: Left endpoint is root: (x+2) on [-2,3] → -2
    try {
        // (x+2) => coefficients: [2, 1]
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
                new double[]{2.0, 1.0});
        double r = solver.solve(1000, f, -2.0, 3.0);
        System.out.printf("VT4 root=%.12f\n", r);
        check("VT4 == -2.0", approx(r, -2.0, 0));
    } catch (Throwable t) { t.printStackTrace(); check("VT4 unexpected
exception", false); }

```

```

    // VT5: near-flat double root at 1: (x-1)^2 (x-
2) on [0.5, 1.5] → ~1
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 1.0, 2.0);
        double r = solver.solve(1000, f, 0.5, 1.5);
        System.out.printf("VT5 root=%.12f, f(root)=%.3e\n", r, f.value
(r));
        check("VT5 ~ 1.0", approx(r, 1.0, 1e-7));
    } catch (Throwable t) { t.printStackTrace(); check("VT5 unexpected
exception", false); }
    // VT6: provide start value explicitly (triggers start-based path)
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(10.0, 1e-6, -3.0);
        double r = solver.solve(1000, f, -
4.0, 11.0, 9.0); // start near 10
        System.out.printf("VT6 root=%.12f, f(root)=%.3e\n", r, f.value
(r));
        check("VT6 ~ 10.0", approx(r, 10.0, 1e-8));
    } catch (Throwable t) { t.printStackTrace(); check("VT6 unexpected
exception", false); }
    // === VT7: yInitial*yMin < 0 → branch: return Laguerre(min, initial)
    // f(x) = (x-1)(x-2)(x-
3), min=0.5 (f<0), initial=1.2 (f>0), max=1.3
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 2.0, 3.0);
        double min = 0.5, initial = 1.2, max = 1.3;
        double yMin = f.value(min), yInitial = f.value(initial);
        System.out.printf("VT7 signs: yMin=%.3f, yInitial=%.3f\n", yMin, yInitial);
        double r = solver.solve(1000, f, min, max, initial);
        System.out.printf("VT7 root=%.12f, f(root)=%.3e\n", r, f.value
(r));
        check("VT7 ~ 1.0", approx(r, 1.0, 1e-10));
    } catch (Throwable t) { t.printStackTrace(); check("VT7 unexpected
exception", false); }
    // === VT8: yInitial*yMax < 0 → branch: return Laguerre(initial, max)

```

```

    //  $f(x) = (x-1)(x-2)(x-3)$ ,  $\min=1.7$  ( $f>0$ ),  $\text{initial}=1.8$  ( $f>0$ ),  $\max=2.5$  ( $f<0$ )
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 2.0, 3.0);
        double min = 1.7, initial = 1.8, max = 2.5;
        double yInitial = f.value(initial), yMax = f.value(max);
        System.out.printf("VT8 signs: yInitial=%.3f, yMax=%.3f\n", yInitial, yMax);
        double r = solver.solve(1000, f, min, max, initial);
        System.out.printf("VT8 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        check("VT8 ~ 2.0", approx(r, 2.0, 1e-10));
    } catch (Throwable t) { t.printStackTrace(); check("VT8 unexpected exception", false); }
    // === VT9: No bracketing → branch: throw new NoBracketingException(min,max,yMin,yMax)
    //  $f(x) = x^2 + 1 > 0$  on  $[-3, 2]$ 
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
                new double[]{1.0, 0.0, 1.0}); //  $1 + x^2$ 
        solver.solve(1000, f, -3.0, 2.0);
        check("VT9 should throw NoBracketingException", false);
    } catch (IllegalArgumentException e) {
        System.out.println("VT9 threw: " + e.getClass().getSimpleName());
    }
    check("VT9 threw (not bracketed)", true);
    } catch (Throwable t) { t.printStackTrace(); check("VT9 wrong exception", false); }
    // === VT10:  $\text{complexSolver.isRoot}(lo,hi,z) == \text{true} \rightarrow \text{return } z.\text{getReal}()$ 
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 2.0);
        double r = solver.solve(1000, f, 0.5, 2.5, 1.8);
        System.out.printf("VT10 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        // Accept either root; both are in  $[0.5, 2.5]$ .
        boolean ok = approx(r, 1.0, 1e-10) || approx(r, 2.0, 1e-10);
    }

```

```

        check("VT10 root inside interval (isRoot true)", ok);
    } catch (Throwable t) { t.printStackTrace(); check("VT10 unexpected exception", false); }
    // === VT11: complexSolver.isRoot(lo,hi,z) == false → solveAll(...) fallback, pick root in [lo,hi]
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 2.0);
        double r = solver.solve(1000, f, 0.5, 2.5, 10.0);
        System.out.printf("VT11 root=%.12f, f(root)=%.3e%n", r, f.value(r));

        boolean ok = approx(r, 1.0, 1e-10) || approx(r, 2.0, 1e-10);
        check("VT11 picked from solveAll fallback", ok);
    } catch (Throwable t) { t.printStackTrace(); check("VT11 unexpected exception", false); }
    // === VT12: isSequence(min, z.getReal(), max) → tolerance gate on imag part (|Im(z)| <= tol or |z| <= fValAcc)
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 1.0, 2.0); // double root at 1
        double r = solver.solve(1000, f, 0.9, 2.1, 1.05);
        System.out.printf("VT12 root=%.12f, f(root)=%.3e%n", r, f.value(r));

        boolean ok = approx(r, 1.0, 1e-7) || approx(r, 2.0, 1e-10);
        check("VT12 tolerance gate accepted", ok);
    } catch (Throwable t) { t.printStackTrace(); check("VT12 unexpected exception", false); }
    // === VT13: force solveAll(...) loop to iterate and pick a later root inside [lo,hi]
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(-5.0, 1.0); // (x+5)(x-1)
        double lo = 0.5, hi = 1.5, start = -10.0; // start near -5 to bias the first found root
        double r = solver.solve(1000, f, lo, hi, start);
        System.out.printf("VT13 root=%.12f, f(root)=%.3e%n", r, f.value(r));

        // Must pick the +1 root from the solveAll List (not the first element)
        check("VT13 picked +1 inside interval via loop", approx(r, 1.0, 1e-10));
    }

```

```

    } catch (Throwable t) { t.printStackTrace(); check("VT13 unexpected
exception", false); }

    // == VT14: Loop completes with NO break (no root in [lo,hi]) → r re
mains NaN
    try {
        //  $f(x) = (x + 5)(x + 2) \rightarrow$  roots at -5 and -
2 (both OUTSIDE  $[0.5, 1.5]$ )
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFu
nction f =
            polyFromRoots(-5.0, -2.0);

        double lo = 0.5, hi = 1.5, start = 10.0; // start far right so  $f(
start) > 0$ ,  $f(lo) > 0$ ,  $f(hi) > 0$ 
        double yLo = f.value(lo), yHi = f.value(hi), yStart = f.value(sta
rt);
        System.out.printf("VT14 signs: yLo=%.3f, yHi=%.3f, yStart=%.3f%n"
, yLo, yHi, yStart);

        double r = solver.solve(1000, f, lo, hi, start);
        System.out.printf("VT14 root=%.12f, f(root)=%s%n", r, Double.isNa
N(r) ? "NaN" : String.format("%.3e", f.value(r)));

        // Expect NO root selected inside  $[lo,hi] \rightarrow$  method returns NaN (L
oop ran to completion, no break).
        check("VT14 returns NaN (no break in loop)", Double.isNaN(r));
    } catch (Throwable t) { t.printStackTrace(); check("VT14 unexpected e
xception", false); }

    // VT15: Left endpoint is root:  $(x+2)$  on  $[-2,3] \rightarrow -2$ 
    try {
        //  $(x+2) \Rightarrow$  coefficients:  $[2, 1]$ 
        org.apache.commons.math4.legacy.analysis.polynomials.Polynomi
alFunction f =
            new org.apache.commons.math4.legacy.analysis.polynomials.
PolynomialFunction(
                new double[]{2.0, 1.0});
        double r = solver.solve(1000, f, -2.0, 3.0);
        System.out.printf("VT4 root=%.12f%n", r);
        check("VT15 == -2.0", approx(r, -2.0, 0));
    } catch (Throwable t) { t.printStackTrace(); check("VT15 unexpect
ed exception", false); }

    // VT16: near-flat double root at 1:  $(x-1)^2 (x-
2)$  on  $[0.5, 1.5] \rightarrow \sim 1$ 

```

```

    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 1.0, 2.0);
        double r = solver.solve(1000, f, 0.5, 1.5);
        System.out.printf("VT16 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        check("VT16 ~ 1.0", approx(r, 1.0, 1e-7));
    } catch (Throwable t) { t.printStackTrace(); check("VT16 unexpected exception", false); }
    // VT17: provide start value explicitly (triggers start-based path)
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(10.0, 1e-6, -3.0);
        double r = solver.solve(1000, f, -
4.0, 11.0, 9.0); // start near 10
        System.out.printf("VT17 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        check("VT17 ~ 10.0", approx(r, 10.0, 1e-8));
    } catch (Throwable t) { t.printStackTrace(); check("VT17 unexpected exception", false); }
    // === VT18: yInitial*yMin < 0 → branch: return Laguerre(min, initial)
    // f(x) = (x-1)(x-2)(x-3), min=0.5 (f<0), initial=1.2 (f>0), max=1.3
    try {
        org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
            polyFromRoots(1.0, 2.0, 3.0);
        double min = 0.5, initial = 1.2, max = 1.3;
        double yMin = f.value(min), yInitial = f.value(initial);
        System.out.printf("VT18 signs: yMin=%.3f, yInitial=%.3f\n", yMin, yInitial);
        double r = solver.solve(1000, f, min, max, initial);
        System.out.printf("VT18 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        check("VT18 ~ 1.0", approx(r, 1.0, 1e-10));
    } catch (Throwable t) { t.printStackTrace(); check("VT18 unexpected exception", false); }
    // === VT19: yInitial*yMax < 0 → branch: return Laguerre(initial, max)

```

```

        // f(x) = (x-1)(x-2)(x-
3), min=1.7 (f>0), initial=1.8 (f>0), max=2.5 (f<0)
        try {
            org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
                polyFromRoots(1.0, 2.0, 3.0);
            double min = 1.7, initial = 1.8, max = 2.5;
            double yInitial = f.value(initial), yMax = f.value(max);
            System.out.printf("VT19 signs: yInitial=%.3f, yMax=%.3f\n", y
Initial, yMax);
            double r = solver.solve(1000, f, min, max, initial);
            System.out.printf("VT19 root=%.12f, f(root)=%.3e\n", r, f.val
ue(r));
            check("VT19 ~ 2.0", approx(r, 2.0, 1e-10));
        } catch (Throwable t) { t.printStackTrace(); check("VT19 unexpect
ed exception", false); }
        // VT20: same cubic, interval targets the middle root ~ 2
        try {
            org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
                polyFromRoots(1.0, 2.0, 3.0);
            double r = solver.solve(1000, f, 1.5, 2.5);
            System.out.printf("VT20 root=%.12f, f(root)=%.3e\n", r, f.val
ue(r));
            check("VT20 ~ 2.0", approx(r, 2.0, 1e-10));
        } catch (Throwable t) { t.printStackTrace(); check("VT20 unexpect
ed exception", false); }
        System.out.println("=== Done LaguerreSolver tests ===");
    }
}
// FT
class FT {
    private static void runCase(
        String name,
        double[] coeffsAsc,    // PolynomialFunction expects ascending p
owers: c0 + c1 x + c2 x^2 + ...
        double min, double max, double start, int maxEval) {
        System.out.println("---- " + name + " ----");
        try {
            org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f =
                new org.apache.commons.math4.legacy.analysis.polynomials.P
olynomialFunction(coeffsAsc);
            LaguerreSolver solver = new LaguerreSolver();

```

```

        double r = solver.solve(maxEval, f, min, max, start);
        double fr = f.value(r);
        System.out.printf("root=%.17g, f(root)=%.3e\n", r, fr);
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() +
" - " + t.getMessage());
    }
}

public void Ftest() {
    System.out.println("=== LaguerreSolver FUZZ: 20 cases ===");
    // FZ01
    runCase("FZ01",
        new double[]{-
1.7728391049103, 0.0000003811202377, 3.901223117, -0.5127780041},
        -2.891027310473, 1.338290177162, -0.448220319711, 200);
    // FZ02
    runCase("FZ02",
        new double[]{3.832023139620673E-12, -
9.110003177, 1.6309011038425763E2, -7.441e-7, 0.390122},
        -0.993001270041, 3.701221180067, 0.118337660422, 300);
    // FZ03
    runCase("FZ03",
        new double[]{4.669201609102990, -
0.000000000000371, 1.00000000000073},
        -1.701112301002, 2.199451788331, 1.944023190044, 1000);
    // FZ04
    runCase("FZ04",
        new double[]{-0.331700220019, 2.840001177, -
1.911772200331, 0.0041200013, -0.000009101, 0.000000021},
        -5.11220399011, -0.22310111701, -2.7700131907, 120);
    // FZ05
    runCase("FZ05",
        new double[]{-0.881177201901, 5.0e-16},
        -9.0, 9.0, 0.0, 20);
    // FZ06
    runCase("FZ06",
        new double[]{-3.7190021109911E5, 0.000312001, 0.00000000079, -
0.01411},
        -20.0, 20.0, -11.313, 400);
    // FZ07
    runCase("FZ07",
        new double[]{0.1122001337, -1.002771, 0.500331, -
0.119002, 0.0100037},
        -3.550022, -0.551, -2.44221011, 300);

```

```
// FZ08
runCase("FZ08",
    new double[]{-1.7976931348623157E308, 1.0, 1.0},
    -2.0, 2.0, 0.5, 5);
// FZ09
runCase("FZ09",
    new double[]{-0.031, 0.0, -0.12, 0.0, 1.6309011038425763E4},
    -3.0, 3.0, 1.337, 200);
// FZ10
runCase("FZ10",
    new double[]{0.00011721, -0.00390221, 0.019771, -0.000321},
    0.781101, 0.789331, 0.780913, 50);
// FZ11
runCase("FZ11",
    new double[]{-2.003311177, 3.117720119, -0.991177201},
    -10.0, 10.0, 4.220331019, 300);
// FZ12
runCase("FZ12",
    new double[]{0.331, -1.22011, 0.8831177, 1.0e-20},
    -1.0, 5.0, 2.00311, 500);
// FZ13
runCase("FZ13",
    new double[]{-1.0e-18, 2.2e-17, -3.1e-16, 4.2e-15, -5.3e-14},
    -100.0, 100.0, 0.00119, 100);
// FZ14
runCase("FZ14",
    new double[]{-0.1122001, 3319.771002},
    -0.0005, 0.0007, 0.0001, 10);
// FZ15
runCase("FZ15",
    new double[]{0.99011, -0.0002117, 0.331, 0.0, -
0.00000009, 1.0},
    -1.1, 0.7, -0.33, 250);
// FZ16
runCase("FZ16",
    new double[]{0.001, -2.00077119, 1.99988221},
    -1.0, 1.0, 0.9991, 80);
// FZ17
runCase("FZ17",
    new double[]{-1.11219, 0.003, 0.0, 2.0},
    -0.75, 0.25, -12.331, 600);
// FZ18
runCase("FZ18",
```

```

        new double[]{-0.771190011, -0.119001, 0.33177001, -
0.0911, 0.01221},
        -4.0, -0.2, -1.337, 320);
    // FZ19
    runCase("FZ19",
        new double[]{3.832023139620673E294, 1.1177e-300, 1.0},
        -1.0, 1.0, 0.0, 3);
    // FZ20
    runCase("FZ20",
        new double[]{-1.0e-30, 0.0, 2.220331177, 1.0e-9},
        -2.0, 2.0, -1.2200331, 700);
    System.out.println("=== Done FUZZ ===");
}
}
// Z3
class Z3 {
    // Build PolynomialFunction from real roots:  $\prod (x - r_i)$ 
    private static org.apache.commons.math4.legacy.analysis.polynomials.Po
ynomialFunction polyFromRoots(
        double... roots) {
        double[] coeff = new double[] { 1.0 }; // ascending powers
        for (double r : roots) {
            double[] next = new double[coeff.length + 1];
            next[0] = -r * coeff[0];
            for (int k = 1; k < next.length - 1; k++) {
                double ck = (k < coeff.length) ? coeff[k] : 0.0;
                double ckm1 = coeff[k - 1];
                next[k] = ckm1 - r * ck;
            }
            next[next.length - 1] = coeff[coeff.length - 1];
            coeff = next;
        }
        return new org.apache.commons.math4.legacy.analysis.polynomials.Polyn
omialFunction(coeff);
    }
    private static void pass(String name) {
        System.out.println("[PASS] " + name);
    }
    private static void fail(String name, String msg) {
        System.out.println("[FAIL] " + name + " :: " + msg);
    }
    private static boolean approx(double a, double b, double tol) {
        if (Double.isNaN(a) || Double.isNaN(b))
            return false;

```

```

    if (Double.isInfinite(a) || Double.isInfinite(b))
        return a == b;
    return Math.abs(a - b) <= tol * Math.max(1.0, Math.max(Math.abs(a), M
ath.abs(b)));
}
private static void expectNullArgument(String name, int maxEval,
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunct
ion f, double min, double max,
    double start) {
    try {
        LaguerreSolver solver = new LaguerreSolver();
        solver.solve(maxEval, f, min, max, start);
        fail(name, "expected NullArgument (null polynomial)");
    } catch (Throwable e) { // avoid multi-
catch conflicts across environments
        if (e instanceof org.apache.commons.math4.legacy.exception.NullArgum
entException
            || e instanceof NullPointerException) {
            pass(name);
        } else {
            fail(name, "wrong exception: " + e.getClass().getSimpleName());
        }
    }
}
private static void expectBadSequence(String name,
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunct
ion f, double min, double max) {
    try {
        LaguerreSolver solver = new LaguerreSolver();
        solver.solve(10, f, min, max);
        fail(name, "expected BadSequence (min>max)");
    } catch (IllegalArgumentException e) {
        pass(name);
    } catch (Throwable t) {
        fail(name, "wrong exception: " + t.getClass().getSimpleName());
    }
}
private static void expectNoBracketing(String name,
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunct
ion f, double min, double max) {
    try {
        LaguerreSolver solver = new LaguerreSolver();
        solver.solve(50, f, min, max);
        fail(name, "expected NoBracketing");
    }
}

```

```

    } catch (IllegalArgumentException e) {
        pass(name);
    } catch (Throwable t) {
        fail(name, "wrong exception: " + t.getClass().getSimpleName());
    }
}

private static void expectTooManyEvaluations(String name,
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f, double min, double max,
    double start, int maxEval) {
    try {
        LaguerreSolver solver = new LaguerreSolver();
        solver.solve(maxEval, f, min, max, start);
        fail(name, "expected TooManyEvaluations");
    } catch (IllegalStateException e) {
        pass(name);
    } catch (Throwable t) {
        fail(name, "wrong exception: " + t.getClass().getSimpleName());
    }
}

private static void expectRootEquals(String name, double expected, double r) {
    if (approx(r, expected, 0))
        pass(name);
    else
        fail(name, "root " + r + " != " + expected);
}

public void Z3test() {
    // 01) c=null -> NullArgument
    expectNullArgument("Z3-01 c=null -> NullArgument", 10, null, -1, 1, 0);
    // 02) min>max -> BadSequence (ex=BadSequence)
    expectBadSequence("Z3-02 min>max -> BadSequence", polyFromRoots(0.0, 0.0, -0.5);
    // 03) not bracketed -> NoBracketing (f(x)=x^2+1 always >0 on [-999,0])
    expectNoBracketing("Z3-03 not bracketed -> NoBracketing",
        new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
            new double[] { 1.0, 0.0, 1.0 }),
            -999.0, 0.0);
    // 04) yMin=0 -> r=min (f(x)=x+1 has root -1 at min)
    try {

```

```

    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { 1.0, 1.0 }); // x+1
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(100, f, -1.0, 2.0);
    expectRootEquals("Z3-04 yMin=0 -> r=min", -1.0, r);
} catch (Throwable t) {
    fail("Z3-04", t.getClass().getSimpleName());
}
// 05) yMax=0 -> r=max (f(x)=x has root 0 at max)
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { 0.0, 1.0 }); // x
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(100, f, -2.0, 0.0);
    expectRootEquals("Z3-05 yMax=0 -> r=max", 0.0, r);
} catch (Throwable t) {
    fail("Z3-05", t.getClass().getSimpleName());
}
// 06) start in interval and yStart=0 -> r=start (f(x)=x, start=0)
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { 0.0, 1.0 }); // x
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(100, f, -1.0, 1.0, 0.0);
    expectRootEquals("Z3-
06 start in interval and yStart=0 -> r=start", 0.0, r);
} catch (Throwable t) {
    fail("Z3-06", t.getClass().getSimpleName());
}
// 07) start*min opposite sign -> r in (min,start] and |f(r)| ≤ fAcc
// f(x)=x, min=-1 (f<0), start=0.5 (f>0), expect r=0 in (min,start]
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { 0.0, 1.0 }); // x
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(100, f, -1.0, 2.0, 0.5);

```

```

    if (r > -1.0 && r <= 0.5 && Math.abs(f.value(r)) <= 1e-6)
        pass("Z3-07 start*min opposite sign");
    else
        fail("Z3-07", "r=" + r);
} catch (Throwable t) {
    fail("Z3-07", t.getClass().getSimpleName());
}
// 08) start*max opposite sign -> r in [start,max) and |f(r)| ≤ fAcc
// f(x)=x-1, start=0 (f<0), max=2 (f>0); expect r=1 in [0,2)
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { -1.0, 1.0 }); // x-1
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(100, f, -1.0, 2.0, 0.0);
    if (r >= 0.0 && r < 2.0 && Math.abs(f.value(r)) <= 1e-6)
        pass("Z3-08 start*max opposite sign");
    else
        fail("Z3-08", "r=" + r);
} catch (Throwable t) {
    fail("Z3-08", t.getClass().getSimpleName());
}
// 09) ∃z in interval: |Im(z)|≤tol or |f(Re z)|≤fAcc -> accept near-real iterate
// (simple real root)
// f(x)=(x-0.5)(x-1.5), interval [-1,2], start 1.1
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(0.5, 1.5);
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(200, f, -1.0, 2.0, 1.1);
    if (approx(r, 0.5, 1e-10) || approx(r, 1.5, 1e-10))
        pass("Z3-09 accept complex iterate");
    else
        fail("Z3-09", "r=" + r);
} catch (Throwable t) {
    fail("Z3-09", t.getClass().getSimpleName());
}
// 10) ∃root[i] in interval: choose from solveAll(...) (start far from bracketed
// root)
// f(x)=(x+5)(x-1), [0.5,1.5], start=-10 -> prefer -
// 5 first; loop selects +1

```

```

try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(-5.0, 1.0);
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(300, f, 0.5, 1.5, -10.0);
    expectRootEquals("Z3-10 choose from solveAll", 1.0, r);
} catch (Throwable t) {
    fail("Z3-10", t.getClass().getSimpleName());
}
// 11) bracketed && no root in interval -> r=NaN (theoretical/infeasible in true
// contract; accept NaN OR throw)
// Using  $f(x)=(x+2)(x+5)$ ; interval  $[0.5, 1.5]$  (no roots in interval).
Not
// bracketed in reality → likely exception.
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(-2.0, -5.0);
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(200, f, 0.5, 1.5, 10.0);
    if (Double.isNaN(r))
        pass("Z3-11 NaN path (if reached)");
    else
        fail("Z3-11", "expected NaN, got r=" + r);
} catch (Throwable t) {
    // Accept exception as this path is logically unreachable under proper
    // bracketing
    pass("Z3-11 acceptable throw (" + t.getClass().getSimpleName() + ")");
}
// 12) evaluations>maxEval -> TooManyEvaluations
//  $f(x)=x(x-1)(x+1)$ , bracketed on  $[-2, 2]$ , start=0, maxEval=1
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(-1.0, 0.0, 1.0);
    expectTooManyEvaluations("Z3-12 TooManyEvaluations", f, -2.0, 2.0, 0.2, 1);
} catch (Throwable t) {
    fail("Z3-12", t.getClass().getSimpleName());
}
// 13)  $y_{Initial} * y_{Min} < 0$  variant (different poly/interval) -> root near 2
try {

```

```

    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { -2.0, 1.0 }); // x-2
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(100, f, 1.5, 3.0, 3.0); // yMin<0, yStart>0
    if (r > 1.5 && r <= 3.0 && approx(r, 2.0, 1e-12))
        pass("Z3-13 start*min opposite sign (variant)");
    else
        fail("Z3-13", "r=" + r);
} catch (Throwable t) {
    fail("Z3-13", t.getClass().getSimpleName());
}
// 14) yInitial*yMax < 0 variant -> root 0 with f(x)=x, start negative, max
// positive
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { 0.0, 1.0 }); // x
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(100, f, -1.0, 0.5, -0.5);
    if (r >= -0.5 && r < 0.5 && approx(r, 0.0, 1e-15))
        pass("Z3-14 start*max opposite sign (variant)");
    else
        fail("Z3-14", "r=" + r);
} catch (Throwable t) {
    fail("Z3-14", t.getClass().getSimpleName());
}
// 15) yMin=0 again (different numbers)
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { 4.0, 1.0 }); // x+4
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(60, f, -4.0, 2.0);
    expectRootEquals("Z3-15 yMin=0 (variant)", -4.0, r);
} catch (Throwable t) {
    fail("Z3-15", t.getClass().getSimpleName());
}
// 16) yMax=0 again (different numbers)
try {

```

```

    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = new org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction(
        new double[] { -2.0, 1.0 }); //  $x-2$ 
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(60, f, -3.0, 2.0);
    expectRootEquals("Z3-16 yMax=0 (variant)", 2.0, r);
} catch (Throwable t) {
    fail("Z3-16", t.getClass().getSimpleName());
}
// 17) double root near 1:  $(x-1)^2(x+2)(x-3)$ , start~1 → acceptance by tolerance
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(1.0, 1.0, -2.0,
        3.0);
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(500, f, 0.2, 1.2, 0.9);
    if (approx(r, 1.0, 1e-7))
        pass("Z3-17 tolerance acceptance near double root");
    else
        fail("Z3-17", "r=" + r);
} catch (Throwable t) {
    fail("Z3-17", t.getClass().getSimpleName());
}
// 18) choose from solveAll but only one in-interval:  $(x+2)(x-3)$ , [2.5,3.5],
// start=-2 → must pick 3
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(-2.0, 3.0);
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(300, f, 2.5, 3.5, -2.0);
    expectRootEquals("Z3-18 solveAll picks 3", 3.0, r);
} catch (Throwable t) {
    fail("Z3-18", t.getClass().getSimpleName());
}
// 19) accept near-real z in [min,max] (variant):  $(x-0.75)(x-1.25)$ 
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(0.75, 1.25);
    LaguerreSolver solver = new LaguerreSolver();
    double r = solver.solve(200, f, 0.5, 1.5, 1.1);
    if (approx(r, 0.75, 1e-10) || approx(r, 1.25, 1e-10))

```

```

    pass("Z3-19 accept z (variant)");
else
    fail("Z3-19", "r=" + r);
} catch (Throwable t) {
    fail("Z3-19", t.getClass().getSimpleName());
}
// 20) TooManyEvaluations (variant): tight budget on cubic with bracketed root
try {
    org.apache.commons.math4.legacy.analysis.polynomials.PolynomialFunction f = polyFromRoots(0.2, 1.4, -0.7);
    expectTooManyEvaluations("Z3-20 TooManyEvaluations (variant)", f, -1.0, 2.0, 0.0, 1);
} catch (Throwable t) {
    fail("Z3-20", t.getClass().getSimpleName());
}
System.out.println("=== Done Z3-style 20 cases ===");
}
}
class test{
    public static void main(String[] args) {
        //VT
        VT t1 = new VT();
        t1.Vtest();
//        // FT
//        FT t2 = new FT();
//        t2.Ftest();
//        // Z3
//        Z3 t3 = new Z3();
//        t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 94.4%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ LaguerreSolver.java	37.9 %	53	87	140
> VT	47.1 %	16	18	34
▼ LaguerreSolver	94.4 %	17	1	18
> ComplexSolver	71.4 %	20	8	28
laguerre(double, double)	83.3 %	5	1	6
real2Complex(double[])	100.0 %	2	0	2
doSolve()	100.0 %	10	0	10
LaguerreSolver()		0	0	0
LaguerreSolver(double)		0	0	0
> test		0	0	0

3. V-Method testing instruction coverage: 74.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ LaguerreSolver.java	44.3 %	1,625	2,045	3,670
> test	70.0 %	7	3	10
▼ LaguerreSolver	74.8 %	187	63	250
> ComplexSolver	87.9 %	276	38	314
real2Complex(double[])	100.0 %	31	0	31
LaguerreSolver()	100.0 %	4	0	4
LaguerreSolver(double)	100.0 %	11	0	11
doSolve()	100.0 %	80	0	80
laguerre(double, double)	100.0 %	61	0	61
> VT	81.5 %	1,155	263	1,418

4. Fuzz testing branch coverage: 61.1%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ LaguerreSolver.java	17.1 %	24	116	140
▼ LaguerreSolver	61.1 %	11	7	18
laguerre(double, double)	16.7 %	1	5	6
> ComplexSolver	46.4 %	13	15	28
doSolve()	80.0 %	8	2	10
real2Complex(double[])	100.0 %	2	0	2
LaguerreSolver()		0	0	0
LaguerreSolver(double)		0	0	0
> FT		0	0	0
> test		0	0	0

5. Fuzz testing instruction coverage: 60.0%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ LaguerreSolver.java	24.7 %	908	2,762	3,670
▼ LaguerreSolver	60.0 %	150	100	250
laguerre(double, double)	45.9 %	28	33	61
> ComplexSolver	61.8 %	194	120	314
doSolve()	95.0 %	76	4	80
real2Complex(double[])	100.0 %	31	0	31
LaguerreSolver()	100.0 %	4	0	4
LaguerreSolver(double)	100.0 %	11	0	11
> test	70.0 %	7	3	10
> FT	100.0 %	557	0	557

6. Z3 branch coverage: 72.2%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ LaguerreSolver.java	34.3 %	48	92	140
> Z3	38.3 %	23	37	60
▼ LaguerreSolver	72.2 %	13	5	18
laguerre(double, double)	16.7 %	1	5	6
> ComplexSolver	42.9 %	12	16	28
real2Complex(double[])	100.0 %	2	0	2
doSolve()	100.0 %	10	0	10
LaguerreSolver()		0	0	0
LaguerreSolver(double)		0	0	0
> test		0	0	0

7. Z3 instruction coverage: 61.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ LaguerreSolver.java	30.9 %	1,134	2,536	3,670
▼ LaguerreSolver	61.6 %	154	96	250
laguerre(double, double)	45.9 %	28	33	61
> ComplexSolver	61.1 %	192	122	314
real2Complex(double[])	100.0 %	31	0	31
LaguerreSolver()	100.0 %	4	0	4
LaguerreSolver(double)	100.0 %	11	0	11
doSolve()	100.0 %	80	0	80
> Z3	69.7 %	781	340	1,121
> test	70.0 %	7	3	10